

روح ماشین

- محدودیت زمان: 10 دقیقه
- امتیاز خط مبنا (baseline): 28.6
- محیط: یک GPU (≈ 16 GB VRAM)، بدون اینترنت
- اندازه راحل: `solution.ipynb` ≤ 20 MB
- فضای ذخیره‌سازی: 5 GB
- مدل‌های از پیش آموزش‌دیده: فقط [bge-base-en-v1.5](#) — یک رمزگذار متن (مدل embedding).

وظیفه یا Task

اتفاقات عجیبی در آرشیو ملی قزاقستان در حال رخ دادن هستند. کتابداران می‌گویند که پایان‌بندی برخی کتاب‌ها قبلاً متفاوت بوده است، اما هیچ‌کس نمی‌تواند آن را ثابت کند — همه نسخه‌ها یکسان‌اند و همه داستان‌ها همچنان معنادارند. از شما به‌عنوان یک پژوهشگر هوش مصنوعی دعوت شده است تا تغییرات را پیدا کنید.



یک قطعه متن با نوشته‌ای به‌قلم انسان آغاز می‌شود، و در نقطه‌ای، بی‌سروصدا، به یک ادامه تولیدشده توسط یک مدل زبانی سوییچ می‌کند. وقتی متن به‌صورت یکپارچه از ابتدا تا انتها خوانده می‌شود، مانند یک نوشته منسجم به نظر می‌رسد — اما جایی در میانه، نویسنده از یک انسان به یک ماشین تغییر کرده است. وظیفه شما این است که آن تغییر را پیدا کنید: اندیس نویسه‌ای که در آن بخش انسانی پایان می‌یابد و بخش ماشینی آغاز می‌شود.

هر نمونه یا سمبل یک رشته منفرد text است. برای هر نمونه دقیقاً یک مرز وجود دارد که همه چیز پیش از آن انسانی است؛ ولی همه چیز از آن به بعد توسط ماشین تولید شده است. دقت کنید که مرز مذکور از یک نمونه به نمونه دیگر می‌تواند متفاوت باشد.

دیتاست

قطعه‌متن‌های (passage) انگلیسی به صورت Plain-Text، هرکدام با یک مرز.

- **بخش A** (پیش از مرز): گزیده‌ای از متن نوشته‌شده به‌قلم انسان.
- **بخش B** (از مرز به بعد): ادامه‌ای تولیدشده توسط یک مدل زبانی، که بر بخش A مشروط شده است.

- هر طرف دست‌کم 180 واژه دارد؛ طول کل ~500-800 واژه است.
- **boundary_char_index** آفست نویسه‌ای (character offset) است که بخش A در آن پایان می‌یابد: یعنی `text[:boundary_char_index]` بخش انسانی و `text[boundary_char_index:].rstrip()` بخش ماشینی است.

آنچه دریافت می‌کنید

شما دو پوشه دریافت می‌کنید (برای مشاهده بهتر جداول، لطفاً به نسخه انگلیسی مراجعه فرمایید):

کاربرد	answers.jsonl ^۱	نمونه‌ها	پوشه
آموزش / ریزتنظیم روش خود	گنجانده شده □	شامل 1,221 نمونه	dataset/train/ پوشه
اجرای پایپ‌لاین و محاسبه محلی امتیاز خود	گنجانده شده (نسخه توسعه) □	شامل 380 نمونه	dataset/test_public/ پوشه

در زمان ارزیابی، پوشه dataset/test_public/ شما با یک مجموعه ارزیابی پنهان جایگزین می‌شود. قالب آن یکسان است، اما **answers.jsonl** را ندارد. نوت‌بوک شما دوباره روی آن اجرا می‌شود و **answers.jsonl** تولیدشده توسط آن امتیازدهی می‌شود.

- لیدربرد عمومی از مجموعه پنهان **test_leaderboard_a** استفاده می‌کند (380 نمونه).
 - رتبه‌بندی نهایی از مجموعه پنهان **test_leaderboard_b** استفاده می‌کند (380 نمونه).
- هر سه مجموعه ارزیابی هم‌اندازه‌اند و از همان توزیع train نمونه‌گیری شده‌اند، بنابراین امتیاز محلی dataset/test_public/ شما برآوردی معقول از امتیاز لیدربرد شما است.

قالب روی دیسک (On-disk format)

```
dataset/train/data.jsonl      # one JSON object per line: {"id": "...", "text": "..."}
dataset/train/answers.jsonl  # {"id": "...", "boundary_char_index": 1842}
dataset/test_public/data.jsonl      # {"id": "...", "text": "..."}
dataset/test_public/answers.jsonl  # dev copy only – ABSENT in the hidden grading set
```

- شناسه‌ها (Id) در **answers.jsonl** با شناسه‌ها در **data.jsonl** مطابقت دارند.
- dataset/train/ (همراه با پاسخ‌ها) هر زمان که آموزش یا ریزتنظیم انجام دهید در دسترس است.

خروجی (قالب ارسال)

شما یک نوت‌بوک منفرد ارسال می‌کنید که باید **solution.ipynb** نامگذاری شده باشد. این نام دقیق فایل الزامی است. هر چیز دیگری بدون اجرا شدن رد می‌شود.

نوت‌بوک شما باید **dataset/test_public/data.jsonl** را بخواند و یک فایل منفرد **answers.jsonl** را در ریشه مخزن (یعنی repository root) بنویسد — در هر خط یک شیء JSON که هر شناسه نمونه را به اندیس نویسه‌ای مرز پیش‌بینی‌شده شما نگاشت می‌کند:

```
{"id": "example_000123", "boundary_char_index": 1790}
{"id": "example_000124", "boundary_char_index": 2450}
```

- **boundary_char_index** باید یک عدد صحیح در بازه `[0, len(text)]` باشد.
- هر شناسه در **dataset/test_public/data.jsonl** باید دقیقاً یکبار ظاهر شود. نمونه‌ای که در **answers.jsonl** وجود نداشته باشد (یا مقداری غیرصحیح / خارج از محدوده داشته باشد)، امتیاز 0 می‌گیرد.

امتیاز دهی

برای هر نمونه، فرض کنید p اندیس پیش‌بینی‌شده شما و t مرز واقعی باشد. امتیاز هر نمونه با فاصله نویسه‌ای به‌صورت نمایی کاهش می‌یابد:

$$\text{score} = \exp\left(-\frac{|p - t|}{\tau}\right) \in (0, 1], \text{ where } \tau = 100.$$

این امر به رفتار زیر برای امتیاز منجر می‌شود:

- $1.0 =$ — دقیقاً نویسه مرز؛
- $0.78 \approx$ — اختلاف 25 نویسه؛ - $0.61 \approx$ — اختلاف 50 نویسه؛
- $0.37 \approx$ — اختلاف 100 نویسه؛
- $0.01 \approx$ — اختلاف 500 نویسه.

امتیاز نهایی، میانگین امتیازهای هر نمونه در میان همه نمونه‌های بخش موردنظر است (در مقیاس 0-100 گزارش می‌شود). دقت کنید که این معیار، نزدیک‌شدن را پاداش می‌دهد، نه صرفاً پاسخ کاملاً دقیق را.

محدودیت‌ها

- **محیط:** یک GPU (≈ 16 GB VRAM)، بدون اینترنت در زمان ارزیابی — مدل مجاز (که در زیر توضیح داده شده است) از قبل فراهم شده است.
- **بودجه زمانی واقعی:** 10 دقیقه برای کل اجرا — این زمان باید هم هرگونه آموزش / ریزتنظیمی را که در زمان ارزیابی انجام می‌دهید و هم استنتاج روی مجموعه ارزیابی را پوشش دهد.
- **مدل ازپیش‌آموزش‌دیده مجاز** — این فهرست جامع است؛ استفاده از هیچ وزن ازپیش‌آموزش‌دیده دیگری مجاز نیست. این مدل از پیش در محیط فراهم شده است (آن را به‌طور معمول بارگذاری کنید، برای مثال `from_pretrained`؛ دقت بفرمایید در زمان ارزیابی اینترنت وجود ندارد):
 - مدل **bge-base-en-v1.5** — یک رمزگذار متن 110M-پارامتری (مدل `embedding`). این مدل `embedding` جمله/قطعه‌متن تولید می‌کند؛ لذا یک مدل زبانی مولد نیست. شما می‌توانید از آن به همان صورتی که هست (ویژگی‌های ثابت) استفاده کنید، یا آن را روی بخش `train` ریزتنظیم کنید (ریزتنظیم کامل در بودجه 16 GB و 10-minute جای می‌گیرد).
- ابزارهای کلاسیک / آماری بدون محدودیت هستند: می‌توانید هر مدل مبتنی بر ویژگی (برای مثال، دسته‌بندی یا رگرسیون‌های `scikit-learn`) را بر مبنای ویژگی‌های `embedding` که خودتان محاسبه می‌کنید بسازید. ولی استفاده از وزن‌های ازپیش‌آموزش‌دیده یادگیری عمیق فقط به مواردی که بالاتر فهرست شدند محدود شده است.